第四节：Session的相关操作

讲师：肖斌

一、Session对象

session用于创建程序和数据库之间的会话，所有对象的载入和保存都需通过session对象 。在Web项目中，一个请求共用一个session对象。

1、创建Session对象的两种方式

```
# 第一种，需要自己提交事务
with Session(bind=engine) as session:
    session.begin()
    try:
        session.add(some_object)
        session.add(some_other_object)
    except:
        session.rollback()
        raise
    else:
        session.commit()

# 第二种， 不需要自己提交事务
with sessionmaker(bind=engine).begin() as session:
    sess.execute()
```

2、新增模型对象操作

对模型对象进行新增，有两种方式：

```
<!--第一种：类SQL方式-->
insert_stmt = insert(User).values(name='name1')
with Session() as sess:
    sess.execute(insert_stmt)
```

```
    sess.commit()

<!--未绑定参数-->
insert_stmt2 = insert(User)
with Session() as sess:
    sess.execute(insert_stmt2,{'name':'name1'})
    sess.commit()

<!--批量-->
with Session() as sess:
    sess.execute(insert_stmt2,[{'name':'name1'},{'name':'name2'}])
    sess.commit()
```

```
<!--第二种:  面向对象方式-->
obj=User(name='name2')
with Session() as sess:
    sess.add(obj)
    sess.commit()

<!--批量-->
obj=User(name='name2')
obj2=User(name='name2')
with Session() as sess:
    sess.add(obj)
    sess.add(obj2)
    # 或者 s.add_all([obj,obj2])
    sess.commit()

 # 批量添加对象
with sessionmaker(engine).begin() as session:
    emp1 = Employee(name='zs', sal=2000, bonus=500, gender=SexValue.MALE)
    emp2 = Employee(name='ls', sal=3000, bonus=400, gender=SexValue.MALE)
    session.add_all((emp1, emp2))
```

3、简单查询操作

1）根据主键查询

```
emp = session.get(Emp, 1)
```

2) 查询整张表的数据

a、返回模型对象

```
statement = select(Employee)
list_emp = session.scalars(statement).all()
for o in list_emp:
    print(o)


# 需要在Employee模型类中增加一个__str__函数
def __str__(self):
    return f'{self.name}, {self.gender.value}, {self.sal}, {self.entry_date}'
```

b、返回row对象，一般用于指定返回的字段

```
statement = select(Employee.name, Employee.sal, Employee.gender)
# list_emp中的元素不是模型对象， 而是包含多个字段只的row对象
list_emp = session.execute(statement).all()
for row in list_emp:
    print(row.name, row.sal, row.gender.value)
```

c、执行原生的SQL，并返回row对象

```
# 执行原生sql
sql_text = text('select id, name, sal, gender from t_emp')
# list_emp中的元素不是模型对象， 而是包含多个字段只的row对象
list_emp = session.execute(sql_text).all()
for row in list_emp:
    print(row)
    print(row.name, row.sal, row.gender)
```

d、执行原生的sql，并返回模型对象

```
# 执行原生sql
sql_text = text('select id, name, sal, gender from t_emp')
# 手动建立映射关系
new_sql = sql_text.columns(Employee.id, Employee.name, Employee.sal,
Employee.gender)
# 得到orm的查询语句
orm_sql = select(Employee).from_statement(new_sql)
```

```
# list_emp中的元素是模型对象
list_emp = session.execute(orm_sql).scalars()
for o in list_emp:
    print(type(o))
    print(o)
```

4、修改操作

1) 先查询出来，再修改属性

```
# 查询，再修改
old_emp = session.get(Emp, 1)
print(old_emp.name)
old_emp.name = '李四'
```

2) 直接根据主键修改

```
# 注意where语句就是条件的意思，其中id前面的Employee必须有
session.execute(update(Employee).where(Employee.id ==
1).values(sal=Decimal(6000), bonus=Decimal(500)))
```

3) 直接根据主机批量修改

```
# 批量修改，必须根据主键来修改
session.execute(update(Employee), [
    {'id': 1, 'name': '张三三'},
    {'id': 2, 'name': '李四四'},
])
```

5、删除操作

1） 先查询，再删除

```
# 先查询出来，再删除
emp = session.get(Employee, 2)
session.delete(emp)
```

2) 直接删除

```
# 删除的数据，由where条件决定
session.execute(delete(Employee).where(Employee.id == 6))
```

6、查询条件

过滤是数据提取的一个很重要的功能，以下对一些常用的过滤条件进行解释，并且这些过滤条件都是只能通过where方法实现的：

1. equals： ==，或者.is_ 函数
2. not equals ：!= 或者 isnot函数
3. like & ilike [不区分大小写]：
4. 在某个集合中存在，in_函数， 或者notin_函数（不存在）
5. And 多条件组合

```
# 查询员工的名字中包含”四“ ，并且基本薪资大于3000的所有员工
where(Employee.name.like('%四%'), Employee.sal > 3000))

where(and_(Employee.name.like('%四%'), Employee.sal > 3000))
```

6. or多条件组合

```
where(or_(Employee.name.like('%四%'), Employee.sal > 3000))
```

7. 聚合函数
   - func.count：统计行的数量。
   - func.avg：求平均值。
   - func.max：求最大值。
   - func.min：求最小值。
   - func.sum：求和。

```
# 查询所以员工的数量
result = session.execute(select(func.count(Employee.id)))
```

8. 分页查询
    a. limit函数：可以限制查询的时候只查询前几条数据。 属top-N查询
    b. offset：可以限制查找数据的时候过滤掉前面多少条。可指定开始查询时的偏移量。

```
result = session.scalars(select(Employee).offset(0).limit(2))
```

9. 排序： order_by函数

```
result =
session.scalars(select(Employee).order_by(Employee.sal.desc()).offset(0).limit(
2))
```

10. 分组查询：group_by 和过滤函数 having

```
# 查询每个性别，各有多少员工
result = session.execute(select(Employee.gender,
func.count(Employee.id)).group_by(Employee.gender))
for o in result:
    # print(type(o))
    print(o)
```